

Notas de Aula

Prof. Walbert Cunha Monteiro

09.05.2026

Capítulo 1	A LINGUAGEM DE PROGRAMAÇÃO JAVA	5
1.1	Componentes Principais	5
1.2	Características	6
1.3	Estrutura de um Programa em Java	7
Capítulo 2	Comentários	9
2.1	Comentário de Linha Única	9
2.2	Comentário de Múltiplas Linhas	9
2.3	Comentário de Documentação (Doc Comment - Javadoc)	9
2.4	Diferença entre comentários comuns e Javadoc	12
Capítulo 3	Variáveis	13
3.1	Tipos Primitivos	13
3.2	Tipos por Referência	14
3.3	Exercícios Resolvidos	16
Capítulo 4	Operadores	19
4.1	Operadores Aritméticos	19
4.2	Operadores de Atribuição	20
4.3	Operadores Relacionais ou de Comparação	21
4.4	Operadores Unários	21
4.5	Operadores Lógicos	23
4.5.1	Tabelas Verdade	24
4.6	Exercícios Resolvidos	26
Capítulo 5	Estruturas Condicionais	27
5.1	If (Simples)	27
5.2	If ... else (Composta)	27
5.3	If Else (Aninhado)	27
5.4	Switch	28
5.5	Exercícios Resolvidos do Capítulo 5	28
Capítulo 6	Laços de Repetição	31
6.1	While	31
6.2	do While	31
6.3	For	32
6.4	Break	33
Capítulo 7	Métodos	35
7.1	Métodos sem Retorno	36
7.1.1	Métodos sem retorno e com parâmetros	36
7.1.2	Métodos sem retorno e sem parâmetros	36

7.2 Métodos com Retorno de Valores.....	37
7.2.1 Métodos com retorno e com parâmetros	37
7.2.2 Métodos com retorno e sem parâmetros	38
7.3 Sobrecarga de Métodos (Method Overloading)	39
7.4 Exercícios Resolvidos do Capítulo 7	40
7.5 Exercícios do Capítulo 7	42
Capítulo 8 Array	45
8.1 Arrays com Tipos Primitivos.....	46
8.2 Arrays com Tipos Não Primitivos	46
8.3 Criar um Array	47
8.3.1 Exibir os elementos de um Array.....	47
8.4 Acessar a elementos do Array	48
8.5 Alterar os de elementos do Array	48
8.6 Inserir elementos em um Array	49
8.7 Remover elementos do Array.....	49
8.8 Pesquisar elementos em um Array	50
8.9 Percorrer um Array com for-each.....	50
8.10 Exercícios Resolvidos do Capítulo 8	50
Capítulo 9 ArrayList	53
9.1 Características	53
9.2 Métodos.....	54
Capítulo 10 HashSet	57
10.1 Características	57
10.2 Métodos.....	57
Capítulo 11 HashMap	59
11.1 Características	59
11.2 Métodos.....	59
Capítulo 12 Fila	61
12.1 Características	61
12.2 Métodos.....	62
12.3 Exemplo	62
Capítulo 13 Stack	63
13.1 Características	63
13.2 Métodos.....	63
Capítulo 14 Exercícios	65

Capítulo 1 A LINGUAGEM DE PROGRAMAÇÃO JAVA

Foi concebida em 1995 por engenheiros da Sun Microsystems. Atualmente, a Oracle Corporation é a proprietária desta tecnologia e a principal mantenedora da plataforma Java. Ao longo dos anos, a linguagem evoluiu para uma plataforma que oferece todo o ambiente necessário para desenvolver e executar aplicações, ganhando destaque como uma importante tecnologia na área da computação, atendendo a diferentes níveis de usuários e operando em diversos dispositivos.

Em função do grande sucesso e aceitação que a linguagem alcançou, foram desenvolvidos diferentes frameworks para facilitar o processo de desenvolvimento de software. Um framework é um conjunto de ferramentas, bibliotecas e convenções que oferece uma estrutura base para o desenvolvimento rápido e padronizado de software, são exemplos de frameworks: Spring, Hibernate, Vaadin, Struts.

1.1 Componentes Principais

Para entender como a linguagem Java funciona, é importante conhecer alguns componentes. Cada um tem uma função específica no processo de criação e execução de programas.

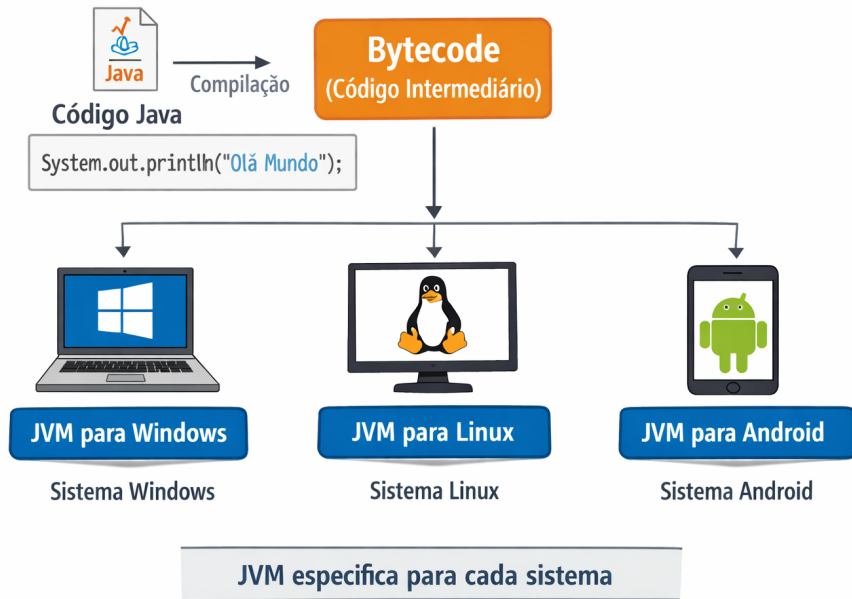
- ❁ **JVM (Java Virtual Machine):** A JVM é a Máquina Virtual Java. Ela é responsável por executar os programas Java. Em vez de rodar diretamente no sistema operacional, o programa é executado dentro da JVM. Isso permite que o mesmo código funcione em diferentes computadores.
- ❁ **JRE (Java Runtime Environment):** A JRE é o ambiente necessário para executar aplicações Java. Ela inclui a JVM e outras bibliotecas essenciais. Se você apenas deseja rodar programas Java, a JRE já é suficiente.
- ❁ **JDK (Java Development Kit):** A JDK é o conjunto completo de ferramentas para desenvolvimento em Java. Ela inclui a JRE e também ferramentas para escrever, compilar e testar programas. Quem deseja programar em Java precisa instalar o JDK.

O Java possui diferentes edições, cada uma voltada para um tipo de aplicação:

- ❁ **Java SE (Standard Edition):** É a base da linguagem. Usada para criar programas simples e aplicações desktop.
- ❁ **Java EE (Enterprise Edition):** Voltada para aplicações maiores, como sistemas web e corporativos.
- ❁ **Java ME (Micro Edition):** Utilizada em dispositivos com recursos limitados, como sistemas embarcados.

Graças à Java Virtual Machine (JVM), que interpreta o código compilado (bytecode), um programa Java pode ser executado em diferentes sistemas operacionais sem precisar ser modificado. Esse princípio é conhecido pela expressão “**Escreva uma vez, execute em qualquer lugar**” (*Write once, run anywhere* – WORA), e representa uma vantagem da linguagem: a portabilidade. A Figura 1.1 ilustra esse processo.

Figura 1.1. O código Java ao ser compilado é transformado em bytecode e executado pela JVM, permitindo que o mesmo programa funcione em diferentes sistemas operacionais.



1.2 Características

A linguagem Java possui ainda várias características que facilitam o desenvolvimento de programas e permitem seu uso em diferentes tipos de aplicações.

- ❁ **Independência de plataforma:** O código Java é compilado em *bytecode* e executado pela JVM. Isso permite que o mesmo programa funcione em diferentes sistemas operacionais, como Windows, Linux e macOS.
- ❁ **Orientação a objetos:** O código é organizado em classes e objetos que interagem entre si. Isso torna o programa mais organizado, reutilizável e fácil de manter.
- ❁ **Gerenciamento automático de memória:** O Java utiliza um mecanismo chamado *Garbage Collector*, que cuida automaticamente da liberação de memória, evitando erros comuns.
- ❁ **Segurança:** Possui mecanismos internos, como verificação de *bytecode* e controle de execução, que ajudam a proteger o sistema contra falhas e códigos maliciosos.
- ❁ **Desempenho:** Mesmo sendo executado pela JVM, o Java utiliza técnicas como compilação Just-In-Time (JIT), que melhora a velocidade de execução dos programas.
- ❁ **Execução de múltiplas tarefas:** O Java permite executar várias tarefas ao mesmo tempo (multithreading), tornando os programas mais eficientes.
- ❁ **Robustez:** É uma linguagem fortemente tipada, o que ajuda a identificar erros ainda durante o desenvolvimento.
- ❁ **Ampla ecossistema:** Possui muitas bibliotecas, ferramentas e documentação, facilitando o aprendizado e o desenvolvimento de aplicações.

1.3 Estrutura de um Programa em Java

Todo programa em Java segue uma estrutura básica. Essa estrutura define como o código deve ser organizado para que o compilador consiga entender e executar corretamente as instruções programadas. Os programas devem ser organizados em uma estrutura hierárquica composta por **classes**, **métodos** e **blocos de código**.

- ❖ **Classe:** É um molde ou modelo que define a estrutura de um programa. Em Java, todo o código deve estar dentro de uma classe.
- ❖ **Métodos:** São blocos de código estruturados dentro de uma classe, projetados para realizar uma tarefa ou mais tarefas específicas.
- ❖ **Blocos de Código:** São estruturas compostas por instruções que compõem a lógica de um programa. Enquanto a instrução é uma única ação executável, o bloco de código é um agrupamento de instruções.
- ❖ **Instruções:** São comandos executáveis, organizados de forma individual ou agrupada com o objetivo de controlar o fluxo do programa.

Exemplo 1.1

```
1 // Classe
2 public class meuPrograma {
3
4     // Método
5     public static void main(String[] args) {
6
7         // Bloco de código
8         System.out.println("Hello World!"); //Instrução
9
10    }
11 }
```

Observação

As **chaves {}** são fundamentais, indicam o início e o fim de cada estrutura. Elas definem onde começa e termina uma classe, um método ou um bloco de código. Um uso incorreto das chaves pode causar erros no programa ou alterar completamente seu funcionamento.

Capítulo 2 Comentários

São anotações, textos explicativos escritos no código do algoritmo ou do programa que **não são executados pelo compilador, são ignorados**. Eles servem como notas para ajudar programadores a entender, organizar e documentar o código, facilitando futuras manutenções e o trabalho em equipe. Os comentários podem ser utilizados para "desativar" temporariamente um trecho do programa.

Observação

Não use comentários para descrever o que a linha de código já diz claramente. Isso apenas aumenta o esforço de leitura sem agregar valor.

2.1 Comentário de Linha Única

É usado para explicações simples e curtas. Neste caso, o trecho do código começa com `//` e todo o texto que segue até o final da linha é afetado.

Exemplo 2.1

```
1 // Variável velocidade é do tipo int
2
3 int velocidade = 80; // Define a velocidade máxima permitida
```

2.2 Comentário de Múltiplas Linhas

Diferentemente dos comentários de linha única, eles abrangem várias linhas consecutivas, geralmente delimitados por caracteres especiais, como `/* ... */`, começam com `/*` e terminam com `*/`.

Exemplo 2.2

```
1 /*
2     Este bloco de código calcula o imposto
3     segundo as regras do ano de 2024.
4 */
5 double imposto = 0.15;
```

2.3 Comentário de Documentação (Doc Comment - Javadoc)

São blocos de texto estruturados de modo que ferramentas possam extraí-los para gerar documentação externa (como HTML ou XML). Ao contrário de comentários comuns (linhas e múltiplas linhas), eles servem para explicar o propósito de elementos como: classes, métodos e parâmetros, facilitando a compreensão de outros desenvolvedores.

Em Java, o comentário de documentação, chamado de **Javadoc** é delimitado por caracteres especiais como `/** ... */`, começam com `/**` e terminam com `*/` e possui tags (palavras) específicas para documentar parâmetros:

- ❁ **@param:** Documenta os parâmetros de um método, indicando o nome de cada parâmetro e sua descrição.
- ❁ **@return:** Descreve o valor retornado pelo método, explicando o que será devolvido após a execução.
- ❁ **@throws ou @exception:** Indica as exceções que o método pode lançar durante sua execução, junto com a condição em que isso ocorre.
- ❁ **@version:** Especifica a versão da classe, interface ou método, útil para o controle da evolução do código.
- ❁ **@since:** Informa a partir de qual versão o elemento (classe, método, etc.) está disponível.
- ❁ **@see:** Cria referências a outros elementos relacionados (classes, métodos ou documentos), facilitando a navegação na documentação.

Exemplo 2.3

```
1 /**
2  * Define o nome de um usuário.
3  * @param nome o nome a ser atribuído ao usuário
4  */
5 public void setName(String nome) {
6     this.nome = nome;
7 }
```

Exemplo 2.4

```
1 /**
2  * Retorna o saldo atual da conta.
3  * @return o valor do saldo disponível
4  */
5 public double getSaldo() {
6     return saldo;
7 }
```

Exemplo 2.5

```
1 /**
2  * Realiza o saque de um valor.
3  * @param valor valor a ser sacado
4  * @throws IllegalArgumentException se o valor for negativo
5  */
6 public void sacar(double valor) {
7     if (valor < 0) {
8         throw new IllegalArgumentException("Valor inválido");
9     }
10    saldo -= valor;
11 }
```

Exemplo 2.6

```
1 /**
2  * Classe que representa um produto.
3  * @version 1.0
4  */
5 public class Produto {
6 }
```

Exemplo 2.7

```
1 /**
2  * Método disponível a partir da versão 2.0.
3  * @since 2.0
4  */
5 public void novoMetodo() {
6     // implementação
7 }
```

Exemplo 2.8

```
1 /**
2  * Calcula o dobro de um número.
3  * @see Math#abs(int)
4  */
5 public int dobro(int x) {
6     return x * 2;
7 }
```

Exemplo 2.9

```
1 /**
2  * Classe que representa uma calculadora simples.
3  * @version 1.0
4  * @since 2026
5  */
6 public class Calculadora {
7
8     /**
9      * Calcula a soma de dois números.
10     * @param a o primeiro número
11     * @param b o segundo número
12     * @return a soma de a e b
13     * @throws IllegalArgumentException se algum valor for negativo
14     * @see Math#addExact(int, int)
15     */
16     public int somar(int a, int b) {
17         if (a < 0 || b < 0) {
18             throw new IllegalArgumentException("Valores não podem ser negativos");
19         }
20         return a + b;
21     }
22 }
```

```
21     }  
22 }
```

2.4 Diferença entre comentários comuns e Javadoc

Os comentários comuns servem apenas para leitura no código, enquanto os comentários de documentação (Javadoc), além de documentar o código, podem gerar documentação oficial.

Capítulo 3 Variáveis

São espaços na memória do computador usados para armazenar informações. Uma variável é como uma "caixinha" onde guardamos um valor para usar depois em um programa.

Cada variável possui um nome e um tipo, que indica o tipo de informação que ela pode armazenar, por exemplo, números ou textos. Em Java, as variáveis podem ser classificadas em dois tipos principais:

- ✿ Tipos primitivos;
- ✿ Tipos por referência.

3.1 Tipos Primitivos

Em Java, os tipos primitivos são utilizados para armazenar valores simples diretamente na memória, como números, caracteres e valores lógicos. Eles são fundamentais na programação, pois representam os dados básicos manipulados pelos programas.

Java possui 8 tipos primitivos, organizados conforme a tabela abaixo:

Tabela 3.1. Tipos Primitivos em Java

Tipo	Descrição	Intervalo de valores	Exemplo
byte	Inteiro pequeno	-128 a 127	byte x = 10;
short	Inteiro curto	-32.768 a 32.767	short x = 100;
int	Inteiro padrão	-2.147.483.648 a 2.147.483.647	int x = 1000;
long	Inteiro grande	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	long x = 10000L;
float	Decimal simples	$\pm 3.4 \times 10^{38}$ (aprox.)	float x = 10.5f;
double	Decimal preciso	$\pm 1.7 \times 10^{308}$ (aprox.)	double x = 10.5;
char	Caractere	0 a 65.535 (Unicode)	char c = 'A';
boolean	Lógico	true ou false	boolean b = true;

Exemplo 3.1

```
1 int idade = 18;  
2 int quantidade = 100;
```

Exemplo 3.2

```
1 double preco = 19.99;  
2 double altura = 1.75;
```

Exemplo 3.3

```
1 char letra = 'A';  
2 char sexo = 'M';
```

Exemplo 3.4

```
1 boolean ativo = true;
2 boolean aprovado = false;
```

Exemplo 3.5

```
1 float temperatura = 36.5f;
2 float nota = 7.5f;
```

Exemplo 3.6

```
1 long populacao = 80000000000L;
```

Exemplo 3.7

```
1 short ano = 2024;
```

Exemplo 3.8

```
1 byte idade = 25;
```

3.2 Tipos por Referência

Os tipos por referência não armazenam o valor diretamente, mas sim uma referência (ou endereço) onde o valor está guardado na memória. Eles são usados para representar dados como textos, listas e objetos.

Tabela 3.2. Tipos por Referência em Java

Tipo	Descrição	Exemplo
String	Cadeia de caracteres	String nome = "João";
Array	Estrutura de vários elementos do mesmo tipo	int[] v = {1,2,3};
Classe (Objeto)	Representa objetos criados pelo programador	Pessoa p = new Pessoa();
Interface	Define comportamentos (contrato)	List l = new ArrayList();
Enum	Conjunto de constantes	Dia d = Dia.SEGUNDA;

Observação

Os tipos de variáveis são classificados como “por referência”, porque a variável aponta para um local na memória onde o dado está armazenado, e não guarda o valor diretamente.

Os tipos por referência não armazenam o valor diretamente, mas sim o endereço na memória onde o valor está. Eles começam com letra maiúscula e são usados para dados mais complexos.

Exemplo 3.9

```
1 String nome = "Maria";  
2 String cidade = "Belem";
```

Exemplo 3.10

```
1 int[] numeros = {1, 2, 3, 4};  
2 String[] nomes = {"Ana", "Joao", "Carlos"};
```

Exemplo 3.11

```
1 class Pessoa {  
2     String nome;  
3 }  
4  
5 Pessoa p = new Pessoa();  
6 p.nome = "Carlos";
```

Exemplo 3.12

```
1 interface Animal {  
2     void emitirSom();  
3 }  
4  
5 class Cachorro implements Animal {  
6     public void emitirSom() {  
7         System.out.println("Latindo");  
8     }  
9 }
```

Exemplo 3.13

```
1 enum Dia {  
2     SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA  
3 }  
4  
5 public class Main {  
6     public static void main(String[] args) {  
7         Dia hoje = Dia.SEGUNDA;  
8         System.out.println(hoje);  
9     }  
10 }
```

Escopo das Variáveis

O escopo define **onde a variável pode ser usada no programa**. Ou seja, o "limite" onde ela existe.

1. Variável Local, é declarada dentro de um método e só pode ser usada dentro dele.

Exemplo 3.14

```
1 public static void main(String[] args) {  
2     int idade = 15; // variável local  
3     System.out.println(idade);  
4 }
```

2. Variável de Classe (ou Global), é declarada fora dos métodos e pode ser usada em toda a classe.

Exemplo 3.15

```
1 public class Exemplo {  
2     static int idade = 15; // variável de classe  
3  
4     public static void main(String[] args) {  
5         System.out.println(idade);  
6     }  
7 }
```

Exemplo de Limite de Escopo:

Exemplo 3.16

```
1 public static void main(String[] args) {  
2     if (true) {  
3         int numero = 10;  
4         System.out.println(numero); // funciona  
5     }  
6  
7 }
```

Nesse caso, a variável `numero` só existe dentro do bloco do `if`.

3.3 Exercícios Resolvidos

Exercício Resolvidos 3.1

```
1 public class ExemploTipos {  
2     public static void main(String[] args) {  
3  
4         int idade = 18;  
5         double altura = 1.75;  
6         char sexo = 'M';  
7         boolean ativo = true;  
8         float nota = 7.5f;  
9         long populacao = 8000000L;  
10        short ano = 2024;  
11        byte nivel = 1;  
12  
13        System.out.println("Idade: " + idade);  
14        System.out.println("Altura: " + altura);  
15    }  
16 }
```

```
15     System.out.println("Sexo: " + sexo);
16     System.out.println("Ativo: " + ativo);
17     System.out.println("Nota: " + nota);
18     System.out.println("Populacao: " + populacao);
19     System.out.println("Ano: " + ano);
20     System.out.println("Nivel: " + nivel);
21 }
22 }
```


Capítulo 4 Operadores

São símbolos ou palavras-chave especiais que indicam ao compilador ou ao interpretador que realize ações específicas, como cálculos matemáticos, comparações ou operações lógicas sobre dados, variáveis ou valores. Sua função principal é manipular dados e controlar o fluxo de um algoritmo. Veremos um conjunto de 5 grupos de operadores comumente utilizados na linguagem de programação Java.

4.1 Operadores Aritméticos

São utilizados para realizar operações aritméticas entre valores e variáveis.

Tabela 4.1. Operadores Aritméticos em Java

Operador	Nome	Descrição	Exemplo
+	Soma	Adiciona dois valores	10 + 5 = 15
-	Subtração	Subtrai um valor do outro	10 - 5 = 5
*	Multiplicação	Multiplica dois valores	10 * 5 = 50
/	Divisão	Divide um valor por outro	10 / 5 = 2
%	Módulo (resto)	Retorna o resto da divisão	10 % 3 = 1

Exemplo 4.1

```
1 public class OperadoresAritmeticos {
2     public static void main(String[] args) {
3         int numero1 = 20, numero2 = 4;
4         // Adição
5         int soma = numero1 + numero2;
6         // Subtração
7         int diferenca = numero1 - numero2;
8         // Multiplicação
9         int multiplicacao = numero1 * numero2;
10        // Divisão
11        int divisao = numero1 / numero2;
12        // Módulo (resto da divisão)
13        int resto = numero1 % numero2;
14
15        System.out.println("Soma: " + soma);
16        System.out.println("Diferença: " + diferenca);
17        System.out.println("Multiplicação: " + multiplicacao);
18        System.out.println("Divisão: " + divisao);
19        System.out.println("Resto: " + resto);
20    }
21 }
```

4.2 Operadores de Atribuição

São operadores que atribuem ou imputam um valor a uma variável e permitem definir um valor inicial ou atualizar o valor já existente, podendo também realizar operações durante a atribuição.

Tabela 4.2. Operadores Básicos de Atribuição em Java

Operador	Nome	Descrição	Exemplo
=	Atribuição simples	Define o valor à direita para a variável à esquerda.	x = 10
+=	Atribuição de adição	Soma o valor à direita ao valor atual da variável e atribui o resultado. Equivalente a x = x + 5.	x += 5
-=	Atribuição de subtração	Subtrai o valor à direita do valor atual da variável. Equivalente a x = x - 3.	x -= 3
*=	Atribuição de multiplicação	Multiplica o valor atual da variável pelo valor à direita. Equivalente a x = x * 2.	x *= 2
/=	Atribuição de divisão	Divide o valor atual da variável pelo valor à direita. Equivalente a x = x / 4.	x /= 4
%=	Atribuição de módulo (resto)	Calcula o resto da divisão entre o valor atual e o valor à direita, atribuindo o resultado. Equivalente a x = x % 3.	x %= 3

Exemplo 4.2

```

1 public class OperadoresAtribuicao {
2     public static void main(String[] args) {
3         int x = 10;
4         System.out.println("Valor inicial: " + x);
5         x += 5;
6         System.out.println("x += 5 -> " + x);
7         x -= 3;
8         System.out.println("x -= 3 -> " + x);
9         x *= 2;
10        System.out.println("x *= 2 -> " + x);
11        x /= 4;
12        System.out.println("x /= 4 -> " + x);
13        x %= 3;
14        System.out.println("x %= 3 -> " + x);
15    }
16 }

```

4.3 Operadores Relacionais ou de Comparação

Este grupo de operadores serve para analisar a relação entre valores ou variáveis. O resultado de uma operação de comparação é sempre um valor booleano: verdadeiro (true) ou falso (false).

Tabela 4.3. Operadores de Comparação em Java

Operador	Nome	Descrição	Exemplo
==	Igual	Verifica se dois valores são iguais	a == b
!=	Diferente	Verifica se os valores são diferentes	a != b
>	Maior que	Verifica se o primeiro valor é maior	a > b
<	Menor que	Verifica se o primeiro valor é menor	a < b
>=	Maior ou igual	Verifica se é maior ou igual	a >= b
<=	Menor ou igual	Verifica se é menor ou igual	a <= b

Exemplo 4.3

```
1 public class Comparacao {
2
3     public static void main(String[] args) {
4
5         int x = 10;
6
7         int y = 5;
8
9         System.out.println(x == y); // Resultado - false
10
11        System.out.println(x != y); // Resultado - true
12
13        System.out.println(x > y); // Resultado - true
14
15        System.out.println(x < y); // Resultado - false
16
17        System.out.println(x >= y); // Resultado - true
18
19        System.out.println(x <= y); // Resultado - false
20    }
21
22 }
```

4.4 Operadores Unários

São operadores que exigem apenas um único operando (variável ou valor) para realizar operações como incremento, decremento, inversão de sinal ou negação lógica. Eles agem diretamente sobre o operando para modificar seu valor ou estado, simplificando tarefas aritméticas e lógicas básicas.

Tabela 4.4. Operadores Unários em Java

Operador	Nome	Descrição	Exemplo
++	Incremento	Aumenta o valor em 1	x++ ou ++x
--	Decremento	Diminui o valor em 1	x- ou -x
+	Positivo	Mantém o valor positivo	+x
-	Negação aritmética	Inverte o sinal do valor	-x
!	Negação lógica	Inverte o valor booleano	!true = false

Exemplo 4.4

```

1 public class ExemploUnarios {
2     public static void main(String[] args) {
3         int x = 5;
4         int y = 5;
5         int z = 5;
6
7         boolean status = true;
8
9         // Incremento
10        System.out.println("Pós-incremento (x++): " + (x++)); // usa e depois
incrementa
11
12        System.out.println("Valor de x após incremento: " + x);
13
14        System.out.println("Pré-incremento (++y): " + (++y)); // incrementa antes de
usar
15
16        // Decremento
17        System.out.println("Pós-decremento (z--): " + (z--)); // usa e depois
decrementa
18
19        System.out.println("Valor de z após decremento: " + z);
20
21        System.out.println("Pré-decremento (--z): " + (--z)); // decrementa antes de
usar
22
23        // Positivo (não altera)
24        int a = -10;
25        System.out.println("Positivo (+a): " + (+a));
26
27        // Negação aritmética
28        int b = 10;
29        System.out.println("Negação (-b): " + (-b));
30
31        // Negação lógica
32        System.out.println("Negação lógica (!status): " + (!status));
33    }
34 }

```

Atenção a condição de **Pré e Pós-Incremento**. São formas de utilizar o operador de incremento (++) e decremento (--). A principal diferença está na ordem em que a operação ocorre:

- ✿ ++x → incrementa primeiro e depois utiliza o valor.
- ✿ x++ → utiliza primeiro e depois incrementa.
- ✿ --x → decrementa primeiro e depois utiliza o valor.
- ✿ x-- → utiliza primeiro e depois decrementa.

Exemplo 4.5

```
1 public class ExemploIncremento {
2     public static void main(String[] args) {
3
4         int x = 5;
5
6         // pré-incremento
7         int a = ++x; // x = 5 + 1 = 6 -> a recebe 6
8
9         // pós-incremento
10        int b = x++; // b recebe 6 -> depois x = 6 + 1 = 7
11
12        System.out.println(x); // Resultado: 7
13
14        System.out.println(a); // Resultado: 6
15
16        System.out.println(b); // Resultado: 6
17    }
18 }
19
20 }
```

Passo a passo:

- ✿ ++x → primeiro, o valor de x é incrementado em 1. Como x valia 5, passa a valer 6. Em seguida, esse novo valor é atribuído à variável **a**. Portanto, a = 6.
- ✿ x++ → primeiro, o valor atual de x é utilizado. Como x valia 6, esse valor é atribuído à variável **b**. Depois disso, x é incrementado em 1, passando a valer 7. Portanto, b = 6 e x = 7.

4.5 Operadores Lógicos

São símbolos utilizados para combinar ou inverter expressões booleanas (true ou false), permitindo explorar a tomada de decisão em estruturas condicionais (por exemplo: if, while). Eles avaliam as condições e retornam um único valor booleano com base nas regras de "E", "OU" e "NÃO".

.

.

.

Tabela 4.5. Operadores Lógicos em Java

Operador	Nome	Descrição	Exemplo
&&	E lógico (AND)	Retorna verdadeiro se ambas as condições forem verdadeiras	<code>x < 5 && x < 10</code>
	OU lógico (OR)	Retorna verdadeiro se pelo menos uma das condições for verdadeira	<code>x < 5 x < 4</code>
!	NÃO lógico (NOT)	Inverte o resultado lógico (true vira false e vice-versa)	<code>!(x < 5 && x < 10)</code>

Exemplo 4.6

```

1 public class ExemploOperadoresLogicos {
2
3     public static void main(String[] args) {
4
5         int x = 7;
6
7         // Operador AND (&&)
8         System.out.println("x > 5 && x < 10: " + (x > 5 && x < 10));
9
10        // Operador OR (||)
11        System.out.println("x < 5 || x < 10: " + (x < 5 || x < 10));
12
13        // Operador NOT (!)
14        System.out.println("!(x > 5 && x < 10): " + !(x > 5 && x < 10));
15    }
16 }

```

** Observação**

As operações envolvendo os operadores lógicos, &&, || e !, seguem as regras da **tabela verdade**. A tabela verdade é a representação das possíveis combinações de valores lógicos (verdadeiro ou falso) e seus respectivos resultados.

4.5.1 Tabelas Verdade

Operador AND (&&) - Uma condição só é verdadeira se todas as condições forem verdadeiras.

Tabela 4.6. Tabela verdade do operador lógico AND (&&)

Condição A	Condição B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

Exemplo 4.7

```

1 public class OperadoresLogicos {
2
3     public static void main(String[] args) {
4
5         // =====
6         // Operador AND (&&)
7         // =====
8         boolean a = true;
9         boolean b = false;
10
11         System.out.println("AND (&&):");
12         System.out.println("true && true = " + (true && true)); // true
13         System.out.println("true && false = " + (true && false)); // false
14         System.out.println("false && true = " + (false && true)); // false
15         System.out.println("false && false = " + (false && false)); // false
16
17     }
18
19 }

```

Operador OR (| |) - Uma condição é verdadeira se pelo menos uma condição for verdadeira.

Tabela 4.7. Tabela verdade do operador lógico OR (| |)

Condição A	Condição B	A B
true	true	true
true	false	true
false	true	true
false	false	false

Exemplo 4.8

```

1 public class OperadoresLogicos {
2
3     public static void main(String[] args) {
4
5         // =====
6         // Operador OR (||)
7         // =====
8         System.out.println("\nOR (||):");
9         System.out.println("true || true = " + (true || true)); // true
10        System.out.println("true || false = " + (true || false)); // true
11        System.out.println("false || true = " + (false || true)); // true
12        System.out.println("false || false = " + (false || false)); // false
13    }
14
15 }

```

Operador NOT (!) - Inverte o valor lógico do resultado.

Tabela 4.8. Tabela verdade do operador lógico NOT (!)

Condição A	!A
true	false
false	true

Exemplo 4.9

```

1 public class OperadoresLogicos {
2     public static void main(String[] args) {
3
4         // =====
5         // Operador NOT (!)
6         // =====
7         System.out.println("\nNOT (!):");
8         System.out.println("!true = " + (!true)); // false
9         System.out.println("!false = " + (!false)); // true
10
11     }
12 }
```

4.6 Exercícios Resolvidos

Exercício Resolvidos 4.1

```

1 public class OperadoresLogicosTeste {
2     public static void main(String[] args) {
3
4         int idade = 20;
5         boolean temCarteira = true;
6         // AND: precisa atender as duas condições
7         if (idade >= 18 && temCarteira) {
8             System.out.println("\nPode dirigir (AND)");
9         }
10        // OR: precisa atender pelo menos uma condição
11        if (idade >= 18 || temCarteira) {
12            System.out.println("Atende pelo menos uma condição (OR)");
13        }
14        // NOT: inverte a condição
15        if (!temCarteira) {
16            System.out.println("Não possui carteira (NOT)");
17        }
18    }
19 }
```

Capítulo 5 Estruturas Condicionais

Servem para tomar decisões no programa. Elas verificam uma condição (verdadeiro ou falso) e executam ações diferentes.

5.1 If (Simples)

Executa o código se a condição for verdadeira.

Exemplo 5.1

```
1 int idade = 18;
2
3 if (idade >= 18) {
4     System.out.println("Maior de idade");
5 }
```

5.2 If ... else (Composta)

Executa um bloco se for verdadeiro e outro se for falso.

Exemplo 5.2

```
1 int idade = 16;
2
3 if (idade >= 18) {
4     System.out.println("Maior de idade");
5 } else {
6     System.out.println("Menor de idade");
7 }
```

5.3 If Else (Aninhado)

Permite testar várias condições.

Exemplo 5.3

```
1 int nota = 5;
2
3 if (nota >= 7) {
4     System.out.println("Aprovado");
5 } else if (nota >= 5) {
6     System.out.println("Recuperacao");
7 } else {
8     System.out.println("Reprovado");
9 }
```

5.4 Switch

Permite selecionar uma alternativa dentre várias, com base no valor de uma variável. Ele é mais indicado quando temos várias comparações de igualdade.

Exemplo 5.4

```
1 int dia = 3;
2
3 switch (dia) {
4     case 1:
5         System.out.println("Domingo");
6         break;
7     case 2:
8         System.out.println("Segunda");
9         break;
10    case 3:
11        System.out.println("Terça");
12        break;
13    default:
14        System.out.println("Outro dia");
15 }
```

5.5 Exercícios Resolvidos do Capítulo 5

Exercício Resolvidos 5.1

```
1 public class ExemploIf {
2
3     public static void main(String[] args) {
4
5         int x = 20;
6         int y = 18;
7
8         if (x > y) {
9             System.out.println("x é maior que y");
10        }
11    }
12 }
```

Exercício Resolvidos 5.2

```
1 public class ParOuImpar {
2
3     public static void main(String[] args) {
4
5         int myNum = 5;
6
7         if (myNum % 2 == 0) {
8             System.out.println(myNum + " é par");
9         }
10    }
11 }
```

```
9         } else {  
10             System.out.println(myNum + " é ímpar");  
11         }  
12     }  
13 }
```

Exercício Resolvidos 5.3

```
1 import javax.swing.JOptionPane;  
2 public class Porta {  
3  
4     public static void main(String[] args) {  
5  
6         int doorCode = Integer.parseInt(  
7             JOptionPane.showInputDialog("Digite o código da porta:")  
8         );  
9  
10        if (doorCode == 1337) {  
11            JOptionPane.showMessageDialog(null, "Código correto. Porta Aberta.");  
12        } else {  
13            JOptionPane.showMessageDialog(null, "Código incorreto. Porta Fechada.");  
14        }  
15    }  
16 }
```

Exercício Resolvidos 5.4

```
1 import javax.swing.JOptionPane;  
2  
3 public class DiaSemana {  
4     public static void main(String[] args) {  
5  
6         int dia = Integer.parseInt(  
7             JOptionPane.showInputDialog("Digite um número de 1 a 7:")  
8         );  
9  
10        if (dia == 1) {  
11            JOptionPane.showMessageDialog(null, "Domingo");  
12        } else if (dia == 2) {  
13            JOptionPane.showMessageDialog(null, "Segunda-feira");  
14        } else if (dia == 3) {  
15            JOptionPane.showMessageDialog(null, "Terça-feira");  
16        } else if (dia == 4) {  
17            JOptionPane.showMessageDialog(null, "Quarta-feira");  
18        } else if (dia == 5) {  
19            JOptionPane.showMessageDialog(null, "Quinta-feira");  
20        } else if (dia == 6) {  
21            JOptionPane.showMessageDialog(null, "Sexta-feira");  
22        } else if (dia == 7) {
```

```
23     JOptionPane.showMessageDialog(null, "Sabado");
24 } else {
25     JOptionPane.showMessageDialog(null, "Valor invalido");
26 }
27 }
28 }
```

Exercício Resolvidos 5.5

```
1 import javax.swing.JOptionPane;
2
3 public class SistemaUsuarios {
4     public static void main(String[] args) {
5
6         String entrada = JOptionPane.showInputDialog(
7             "Digite o tipo de usuario:\n1 - Aluno\n2 - Professor\n3 - Coordenador"
8         );
9
10        int opcao = Integer.parseInt(entrada);
11
12        switch (opcao) {
13            case 1:
14                JOptionPane.showMessageDialog(null, "Acesso ao sistema de estudos");
15                break;
16            case 2:
17                JOptionPane.showMessageDialog(null, "Acesso ao sistema de ensino");
18                break;
19            case 3:
20                JOptionPane.showMessageDialog(null, "Acesso completo ao sistema");
21                break;
22            default:
23                JOptionPane.showMessageDialog(null, "Usuario invalido");
24        }
25    }
26 }
```

Capítulo 6 Laços de Repetição

6.1 While

É uma estrutura de repetição que executa um bloco de código enquanto uma condição for verdadeira. Ou seja, ele repete instruções até que a condição se torne falsa.

Exemplo 6.1

```
1 while (condicao) {  
2     // bloco de codigo a ser executado  
3 }
```

A **condição** é verificada antes de cada execução do laço. Enquanto a condição for verdadeira, o bloco de código será executado. Quando a condição se tornar falsa, o laço é encerrado.

Exemplo 6.2

```
1 public class ExemploWhile {  
2     public static void main(String[] args) {  
3  
4         int i = 1;  
5  
6         while (i <= 5) {  
7             System.out.println("Valor: " + i);  
8             i++;  
9         }  
10    }  
11 }
```

Neste exemplo, a variável *i* é inicializada com o valor 1. O laço `while` verifica se *i* <= 5. Enquanto essa condição for verdadeira, o programa imprime o valor de *i* e, em seguida, incrementa a variável em 1. Esse processo se repete até que *i* se torne maior que 5, momento em que a condição se torna falsa e o laço é encerrado.

Observação

Ao utilizar o laço `while`, é fundamental ter cuidado com a definição da condição de parada, pois um erro nessa lógica pode resultar em um laço infinito. Isso ocorre quando a condição permanece sempre verdadeira, fazendo com que o bloco de código seja executado indefinidamente.

6.2 do While

É uma estrutura de repetição que executa um bloco de código pelo menos uma vez, pois a condição é verificada após a execução.

Exemplo 6.3

```
1 do {  
2     // bloco de código a ser executado  
3 } while (condicao);
```

O bloco de código é executado pelo menos uma vez antes da verificação da condição. A **condição** é verificada após a execução do bloco. Se a condição for verdadeira, o laço continua; caso contrário, é encerrado.

Exemplo 6.4

```
1 public class ExemploDoWhile {  
2     public static void main(String[] args) {  
3  
4         int i = 1;  
5  
6         do {  
7             System.out.println("Valor: " + i);  
8             i++;  
9         } while (i <= 5);  
10    }  
11 }
```

Neste exemplo, o bloco de código dentro do do é executado inicialmente, independentemente da condição. Após a execução, a condição `i <= 5` é verificada. Se for verdadeira, o laço continua executando. Caso contrário, ele é encerrado. Assim, o valor de `i` é impresso e incrementado até que ultrapasse 5.

6.3 For

É uma estrutura de repetição utilizada quando sabemos previamente quantas vezes um bloco de código deve ser executado. Ela reúne em uma única linha a inicialização, a condição e o incremento.

Exemplo 6.5

```
1 for (inicializacao; condicao; incremento) {  
2     // bloco de código a ser executado  
3 }
```

- ❁ Inicialização, é executada uma única vez antes do início do laço.
- ❁ Condição, define se o bloco de código será executado.
- ❁ Incremento é executado após cada repetição do laço.

Exemplo 6.6

```
1 public class ExemploFor {  
2     public static void main(String[] args) {  
3
```

```
4     for (int i = 1; i <= 5; i++) {  
5         System.out.println("Valor: " + i);  
6     }  
7 }  
8 }
```

Neste exemplo, a variável `i` é inicializada com o valor 1. A condição `i <= 5` é verificada antes de cada execução. Enquanto essa condição for verdadeira, o valor de `i` é exibido. Após cada repetição, a variável é incrementada em 1. O laço é encerrado quando `i` se torna maior que 5.

Observação

O laço `for` é indicado quando o número de repetições é conhecido, pois concentra o controle da variável em uma única estrutura, reduzindo erros como esquecer o incremento.

6.4 Break

O comando `break` é utilizado para interromper imediatamente a execução de um laço de repetição ou de uma estrutura, por exemplo `switch`.

Exemplo 6.7

```
1 public class ExemploBreak {  
2     public static void main(String[] args) {  
3  
4         for (int i = 1; i <= 10; i++) {  
5             if (i == 5) {  
6                 break;  
7             }  
8             System.out.println(i);  
9         }  
10    }  
11 }
```

Neste exemplo, o laço `for` percorre valores de 1 a 10. Quando a variável `i` atinge o valor 5, o comando `break` é executado, interrompendo o laço. Dessa forma, apenas os valores de 1 até 4 são exibidos.

Exemplo 6.8

```
1 int dia = 2;  
2  
3 switch (dia) {  
4     case 1:  
5         System.out.println("Domingo");  
6         break;  
7     case 2:  
8         System.out.println("Segunda");  
9         break;  
10    default:
```

```
11     System.out.println("Outro dia");  
12 }
```

No comando `switch`, o `break` evita que os próximos casos sejam executados após encontrar a condição correspondente.

Observação

O uso do `break` é importante para controlar o fluxo de execução, evitando execuções desnecessárias e garantindo que o programa saia do laço ou da estrutura no momento adequado.

Capítulo 7 Métodos

Definição 7.1

É um bloco de código estruturado dentro de uma classe, projetado para executar uma tarefa específica e ser reutilizado.

Ele define o comportamento de um objeto, podendo receber parâmetros (entrada), processar informações e retornar um resultado, facilitando a organização, manutenção e leitura do código.

Sua estrutura simplificada é formada por:

Exemplo 7.1

```
1 "qualificador" "tipo-de-retorno" "nome-metodo"([lista-de-parametros]){  
2     //código do corpo  
3 }
```

Qualificador: Define quem pode acessar o método (sua visibilidade).

- ❖ **public:** pode ser usado por qualquer classe.
- ❖ **private:** pode ser usado apenas dentro da própria classe.
- ❖ **protected:** pode ser usado pela própria classe, subclasses e classes do mesmo pacote.

Tipo de Retorno: Indica o tipo de dado que o método devolve após concluir sua execução.

- ❖ Quando o método não retorna nenhum valor, utiliza-se a palavra **void**.
- ❖ O valor retornado pode ser de um tipo primitivo: **byte**, **short**, **int**, **long**, **float**, **double**, **char** e **boolean**; um **array** ou até mesmo um objeto.

Nome do Método: É o nome que identifica o método.

- ❖ Deve começar com letra minúscula.
- ❖ Se tiver mais de uma palavra, as próximas começam com letra maiúscula (ex: **imprimeTexto**).
- ❖ Não pode conter espaços.
- ❖ Deve ser um nome claro, que indique o que o método faz.

Lista de Parâmetros: São os dados que o método pode receber para executar sua tarefa, também são denominados **argumentos**).

- ❖ São opcionais (o método pode ou não ter parâmetros).
- ❖ Pode haver mais de um parâmetro (argumento).
- ❖ o nome do método acrescido de seus parâmetros é denominado **assinatura do método**.
- ❖ Funcionam como valores de entrada para o método.
- ❖ Esses valores podem ser usados no processamento para gerar resultados.

De acordo com o **tipo de retorno**, os métodos podem ser classificados em: **sem retorno** ou **com retorno**. Além disso, ambos podem ser **com parâmetros** ou **sem parâmetros**.

7.1 Métodos sem Retorno

Métodos sem retorno são aqueles que executam uma ação, mas não devolvem nenhum valor ao final. Por isso, utilizam a palavra-chave `void`. A estrutura geral é:

Exemplo 7.2

```
1 "qualificador" "void" "nomeDoMetodo"([listaDeParametros]){  
2     // código do corpo  
3 }
```

Os métodos sem retorno de valores podem ser definidos **com parâmetros** ou **sem parâmetros**.

7.1.1 Métodos sem retorno e com parâmetros

São métodos que recebem dados (parâmetros), realizam uma operação e exibem o resultado, mas não retornam valores.

Exemplo 7.3

```
1 public class MetodosSemRetorno {  
2  
3     public static void main(String[] args) {  
4         imprimirTexto("Olá");  
5         somar(10, 20);  
6         verificarPar(5);  
7     }  
8  
9     public static void imprimirTexto(String texto) {  
10        System.out.println(texto);  
11    }  
12  
13    public static void somar(int a, int b) {  
14        System.out.println(a + b);  
15    }  
16  
17    public static void verificarPar(int numero) {  
18        if (numero % 2 == 0) {  
19            System.out.println("Par");  
20        } else {  
21            System.out.println("Ímpar");  
22        }  
23    }  
24 }
```

7.1.2 Métodos sem retorno e sem parâmetros

São métodos que não recebem dados de entrada para seu processamento.

Exemplo 7.4

```
1 public class MetodosSemRetorno {
2     public static void main(String[] args) {
3         imprimir();
4     }
5
6     public static void imprimir() {
7         System.out.println("Aprendendo Java");
8     }
9 }
```

7.2 Métodos com Retorno de Valores

Os métodos com retorno de valores devolvem um valor ao final de sua execução. Isso é feito por meio da instrução `return`. Estes seguem a seguinte estrutura:

Exemplo 7.5

```
1 "qualificador" "tipoDoRetorno" "nomeDoMetodo"([listaDeParametros]){
2     // código do corpo
3     return valorDeRetorno;
4 }
```

Essa estrutura é semelhante à dos métodos sem retorno, com a diferença de que não há a palavra **void** e o valor é devolvido por meio da instrução `return` ao final do método.

** Observação**

Após o `return`, deve ser indicado o valor a ser retornado, geralmente uma variável ou uma expressão. O uso de parênteses após a instrução `return` é opcional. Estes métodos podem ser construídos com ou sem parâmetros.

Os métodos com retorno de valores podem ser definidos **com parâmetros** ou **sem parâmetros**.

7.2.1 Métodos com retorno e com parâmetros

São métodos que recebem valores de entrada (parâmetros), processam esses valores e retornam um resultado.

Exemplo 7.6

```
1 public static int somar(int a, int b) {
2     return a + b;
3 }
```

Exemplo 7.7

```
1 public static int quadrado(int numero) {  
2     return numero * numero;  
3 }
```

Exemplo 7.8

```
1 public static String verificarPar(int numero) {  
2     if (numero % 2 == 0) {  
3         return "Par";  
4     } else {  
5         return "Ímpar";  
6     }  
7 }
```

Exemplo 7.9

```
1 public static void main(String[] args) {  
2     System.out.println(somar(5, 3));  
3     System.out.println(quadrado(4));  
4     System.out.println(verificarPar(7));  
5 }
```

7.2.2 Métodos com retorno e sem parâmetros

São métodos que não recebem dados de entrada, mas ainda assim retornam um valor.

Exemplo 7.10

```
1 public static int obterNumero() {  
2     return 10;  
3 }
```

Exemplo 7.11

```
1 public static String saudacao() {  
2     return "Olá, seja bem-vindo!";  
3 }
```

Exemplo 7.12

```
1 public static void main(String[] args) {  
2     System.out.println(obterNumero());  
3     System.out.println(saudacao());  
4 }
```

7.3 Sobrecarga de Métodos (Method Overloading)

A sobrecarga de métodos ocorre quando criamos vários métodos com o mesmo nome, mas com parâmetros diferentes. Isso permite reutilizar o mesmo nome de método para realizar tarefas semelhantes, variando apenas a quantidade ou o tipo de dados recebidos. Por que usar sobrecarga?

A sobrecarga de métodos facilita:

- ✿ a organização do código;
- ✿ a reutilização de nomes de métodos;
- ✿ a leitura e compreensão do programa.

Exemplo 7.13

```
1 public class ExemploSobrecarga {  
2  
3     public static void main(String[] args) {  
4         System.out.println(somar(2, 3));  
5         System.out.println(somar(2, 3, 4));  
6     }  
7  
8     public static int somar(int a, int b) {  
9         return a + b;  
10    }  
11  
12    public static int somar(int a, int b, int c) {  
13        return a + b + c;  
14    }  
15 }
```

Exemplo 7.14

```
1 public class ExemploSobrecarga2 {  
2  
3     public static void main(String[] args) {  
4         System.out.println(somar(5, 2));  
5         System.out.println(somar(5.5, 2.5));  
6     }  
7  
8     public static int somar(int a, int b) {  
9         return a + b;  
10    }  
11  
12    public static double somar(double a, double b) {  
13        return a + b;  
14    }  
15  
16 }
```

Regras importantes:

- ✿ Os métodos devem ter o mesmo nome.

- ✿ Devem possuir parâmetros diferentes (quantidade ou tipo).
- ✿ Apenas mudar o tipo de retorno não caracteriza sobrecarga.

7.4 Exercícios Resolvidos do Capítulo 7

Exercício Resolvidos 7.1

```
1 public class MetodosSemRetorno {
2
3     public static void main(String args[]) {
4         imprimir();
5         imprimirTexto("Ola");
6         quadrado(10);
7         somar(10, 20);
8         maior(10, 20, 30);
9         sexo('F');
10    }
11
12    public static void imprimir() {
13        System.out.println("Aprendendo a Linguagem Java");
14    }
15
16    public static void imprimirTexto(String texto) {
17        System.out.println(texto);
18    }
19
20    public static void somar(int a, int b) {
21        System.out.println(a + b);
22    }
23
24    public static void quadrado(int a) {
25        System.out.println(Math.pow(a, 2));
26    }
27
28    public static void maior(int a, int b, int c) {
29        System.out.println(Math.max(c, Math.max(a, b)));
30    }
31
32    public static void sexo(char c) {
33        if (c == 'F') {
34            System.out.println("Feminino");
35        } else if (c == 'M') {
36            System.out.println("Masculino");
37        } else {
38            System.out.println("Sexo desconhecido");
39        }
40    }
41 }
```

Exercício Resolvidos 7.2

```
1 public class MetodosComRetorno {
2
3     public static void main(String[] args) {
4         System.out.println(somar(10, 20));
5         System.out.println(quadrado(5));
6         System.out.println(maior(3, 8));
7     }
8
9     public static int somar(int a, int b) {
10         return a + b;
11     }
12
13     public static int quadrado(int numero) {
14         return numero * numero;
15     }
16
17     public static int maior(int a, int b) {
18         if (a > b) {
19             return a;
20         } else {
21             return b;
22         }
23     }
24 }
25
26 }
```

Exercício Resolvidos 7.3

```
1 public class AreaComSobrecarga {
2
3     public static void main(String args[]) {
4         System.out.println("Área de um quadrado: " + area(3));
5         System.out.println("Área de um retângulo: " + area(3, 2));
6         System.out.println("Área de um cubo: " + area(3, 2, 5));
7     }
8
9     public static double area(int x) {
10         return (x * x);
11     }
12
13     public static double area(int x, int y) {
14         return (x * y);
15     }
16
17     public static double area(int x, int y, int z) {
18         return (x * y * z);
19     }
20 }
```

7.5 Exercícios do Capítulo 7

Exercício 7.1

Questão – Método sem retorno com parâmetros

Crie um método chamado `dobro(int numero)` que receba um número inteiro e mostre na tela o dobro desse valor.

Exercício 7.2

Questão – Método sem retorno sem parâmetros

Crie um método chamado `mensagem()` que exiba na tela a frase: *"Bem-vindo ao Java!"*. Chame esse método no `main`.

Exercício 7.3

Questão - Método com retorno com parâmetros

Crie um método chamado `multiplicar(int a, int b)` que receba dois números e retorne o resultado da multiplicação.

Exercício 7.4

Questão - Método com retorno sem parâmetros

Crie um método chamado `obterNumero()` que retorne o número 100. Mostre esse valor no `main`.

Exercício 7.5

Questão Sobrecarga de métodos (soma)

Implemente dois métodos com o nome `somar`: um que receba dois números inteiros e outro que receba três números inteiros. Em seguida, chame os dois métodos no `main` e exiba os resultados.

Exercício 7.6

Questão - Sobrecarga de métodos (tipos diferentes)

Implemente dois métodos com o nome `multiplicar`: um que receba dois valores do tipo `int` e outro que receba dois valores do tipo `double`. No método `main`, chame ambos e apresente os resultados.

Exercício 7.7**Cálculo de Frete com Taxa Administrativa**

A transportadora **LogExpress** necessita de um novo módulo para seu sistema de faturamento. Sua tarefa é desenvolver um programa em Java que calcule o custo final de transporte de mercadorias.

? Requisitos de Negócio:

- 1. Entrada de Dados:** O programa deve solicitar ao usuário o *peso da carga* (P) em quilogramas e o *valor do frete por quilo* (V_k).
- 2. Regra de Cálculo:** O valor total do frete (T) é composto pelo custo variável somado a uma taxa administrativa fixa de **R\$ 25,00**. A fórmula a ser aplicada é:

$$T = (P \times V_k) + 25.00$$

- 3. Validação:** O sistema não deve aceitar valores negativos ou nulos para o peso ou para o valor unitário. Caso ocorra, uma mensagem de erro deve ser exibida.
- 4. Saída de Dados:** O resultado final deve ser exibido com o prefixo da moeda brasileira (R\$) e formatado obrigatoriamente com duas casas decimais.

Exercício 7.8

Questão - Sobrecarga de Métodos Crie uma classe chamada `CaixaEletronico` que gerencie o saldo de uma conta. A classe deve conter:

- ✿ Um atributo `saldo`.
- ✿ Um método `sacar(double valor)`: Este é o saque para **clientes comuns**. Ele deve descontar o valor do saque mais uma taxa fixa de **0,5%**.
- ✿ Uma sobrecarga do método `sacar(double valor, boolean ehEspecial)`: Se o parâmetro `ehEspecial` for `true`, a taxa aplicada deve ser de apenas **0,1%**.
- ✿ Um método para exibir o saldo atual.

Capítulo 8 Array

Definição 8.1

Um array é uma estrutura de dados que permite armazenar um conjunto de valores em uma única variável.

O array é composto por diferentes elementos:

- ❖ **Tipo de dado**, que define qual tipo de valor o array pode armazenar, como números ou textos. Todos os valores devem ser do mesmo tipo, caracterizando uma estrutura homogênea;
- ❖ **Elementos**, que são os valores armazenados no array. Cada elemento ocupa uma posição específica na estrutura. Essas posições são identificadas pelos índices, que indicam a localização de cada elemento dentro do array, sempre iniciando em zero;
- ❖ **Tamanho** representa a quantidade total de elementos que o array pode armazenar. Dessa forma, o array organiza os dados de maneira estruturada, permitindo acesso rápido por meio dos índices.

Exemplo 8.1

		Índices						
		0	1	2	3	4	5	6
v		-8	4	21	23	54	67	90
Elementos (valores)								
Tipo de dados: int								
Tamanho: 7								

A estrutura de um array em Java segue um padrão simples: tipo dos dados, nome do array e tamanho ou valores iniciais.

Exemplo 8.2

```
1 public class ArrayObjeto {
2     public static void main(String[] args) {
3
4         // Declaração e criação do array
5         tipoDoArray[] nomeDoArray = new tipoDoArray[tamanho];
6         // tipoDoArray - define o tipo dos elementos (int, String, etc.)
7         // nomeDoArray - nome da variável (identificador)
8         // tamanho - quantidade de posições no array
9
10        // Inicialização direta com valores
11        tipoDoArray[] nomeDoArray = {valor1, valor2, valor3, ...};
12        // valor1, valor2... - elementos armazenados no array
13        // os índices começam em 0 automaticamente
```

8.1 Arrays com Tipos Primitivos

Arrays de tipos primitivos armazenam os valores diretamente na memória, de forma contínua. Cada posição do array contém o próprio dado.

Os principais tipos primitivos em Java são: `int`, `double`, `float`, `char`, `boolean`, `byte`, `short` e `long`.

Exemplo 8.3

```
1 public class ArrayPrimitivo {
2     public static void main(String[] args) {
3         int[] numeros = {10, 20, 30, 40};
4
5         for (int i = 0; i < numeros.length; i++) {
6             System.out.println(numeros[i]);
7         }
8
9         // Array armazena os valores diretamente
10        // {10, 20, 30, 40}
11    }
12 }
```

8.2 Arrays com Tipos Não Primitivos

Arrays de tipos não primitivos armazenam referências para objetos na memória, e não os dados diretamente. Cada posição do array aponta para um objeto criado em outro local da memória.

Os principais tipos não primitivos incluem: `String`, `Integer`, `Double`, `Boolean`, além de classes criadas pelo programador.

Exemplo 8.4

```
1 public class ArrayObjeto {
2     public static void main(String[] args) {
3
4         String[] nomes = {"Pedro", "Maria", "Juca"};
5
6         for (String nome : nomes) {
7             System.out.println(nome);
8         }
9
10        // Array armazena referências para objetos String
11        // {"Pedro", "Maria", "Juca"}
12    }
13
14 }
```

8.3 Criar um Array

A criação de um objeto array envolve a definição do tipo de dado, do nome da variável e do número de posições que ele irá armazenar.

Exemplo 8.5

```
1 public class ExemploArray {
2     public static void main(String[] args) {
3
4         // Criando e inicializando um array
5         int[] numeros = {10, 20, 30, 40};
6
7         // Acessando elementos do array
8         System.out.println("Primeiro elemento: " + numeros[0]);
9         System.out.println("Segundo elemento: " + numeros[1]);
10
11        // Mostrando o tamanho do array
12        System.out.println("Tamanho do array: " + numeros.length);
13    }
14 }
```

Exemplo 8.6

```
1 public class ExemploArray {
2
3     public static void main(String[] args) {
4
5         // Tipo: int (números inteiros)
6         // Nome do array: numeros
7         // Tamanho: 4 posições
8         int[] numeros = new int[4];
9
10        // Atribuindo valores (elementos)
11        numeros[0] = 10; // índice 0
12        numeros[1] = 20; // índice 1
13        numeros[2] = 30; // índice 2
14        numeros[3] = 40; // índice 3
15
16        // Exibindo os valores
17        for (int i = 0; i < numeros.length; i++) {
18            System.out.println("Índice " + i + ": " + numeros[i]);
19        }
20    }
21 }
```

8.3.1 Exibir os elementos de um Array

Para acessar todos os elementos de um array, utilizamos estruturas de repetição, como o laço for.

Exemplo 8.7

```
1 public class PercorrerArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // Percorrendo o array
7         for (int i = 0; i < numeros.length; i++) {
8             System.out.println(numeros[i]);
9         }
10
11     }
12 }
```

8.4 Acessar a elementos do Array

O acesso a um elemento é feito por meio de índices, que indicam a posição de cada elemento. Em Java, os índices sempre começam em zero.

Exemplo 8.8

```
1 public class AcessoArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // Acessando elementos
7         System.out.println(numeros[0]); // 10
8         System.out.println(numeros[2]); // 30
9     }
10 }
```

8.5 Alterar os de elementos do Array

Os elementos de um array podem ser alterados acessando uma posição específica e atribuindo um novo valor.

Exemplo 8.9

```
1 public class AlteracaoArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // Alterando um valor
7         numeros[1] = 50;
8
9         // Percorrendo o array
```

```
10     for (int i = 0; i < numeros.length; i++) {
11         System.out.println(numeros[i]);
12     }
13
14     // Resultado - Novo array
15     // {10, 50, 30, 40}
16 }
17 }
```

8.6 Inserir elementos em um Array

A inserção em arrays é limitada, pois seu tamanho é fixo. Assim, é possível apenas substituir valores em posições já existentes.

Exemplo 8.10

```
1 public class InsercaoArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // Inserindo (substituindo) valor
7         numeros[2] = 99;
8
9         // Novo array
10        // {10, 20, 99, 40}
11    }
12 }
```

8.7 Remover elementos do Array

Arrays não permitem remoção direta de elementos. Uma alternativa é substituir o valor por outro.

Exemplo 8.11

```
1 public class RemocaoArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // "Removendo" elemento
7         numeros[1] = 0;
8
9         // Novo array
10        // {10, 0, 30, 40}
11    }
12 }
```

8.8 Pesquisar elementos em um Array

A pesquisa consiste em procurar um valor dentro do array, normalmente utilizando um laço de repetição.

Exemplo 8.12

```
1 public class PesquisaArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5         int busca = 30;
6
7         for (int i = 0; i < numeros.length; i++) {
8             if (numeros[i] == busca) {
9                 System.out.println("Encontrado na posição: " + i);
10            }
11        }
12    }
13 }
```

8.9 Percorrer um Array com for-each

O laço for-each é uma forma simplificada de percorrer um array, acessando diretamente cada elemento, sem utilizar índices.

Exemplo 8.13

```
1 public class PercorrerArrayForEach {
2     public static void main(String[] args) {
3
4         String[] cars = {"BYD", "BMW", "Ford", "Hyundai"};
5
6         // Percorrendo o array com for-each
7         for (String car : cars) {
8             System.out.println(car);
9         }
10    }
11 }
12 }
```

8.10 Exercícios Resolvidos do Capítulo 8

Para percorrer um array e realizar operações com seus elementos, utilizamos estruturas de repetição, como o laço for. Neste exemplo, somamos todos os valores armazenados no array.

Exercício Resolvidos 8.1

```
1 public class SomaArray {  
2     public static void main(String[] args) {  
3  
4         int[] numbers = {1, 5, 10, 25};  
5         int sum = 0;  
6  
7         // Percorrendo o array e somando os elementos  
8         for (int i = 0; i < numbers.length; i++) {  
9             sum += numbers[i];  
10        }  
11  
12        System.out.println("A soma é: " + sum);  
13    }  
14 }
```


Capítulo 9 ArrayList

É uma classe da biblioteca `java.util` que implementa uma lista dinâmica, ou seja, um vetor que pode crescer ou diminuir automaticamente conforme a necessidade. Diferentemente de arrays tradicionais, o `ArrayList` não possui tamanho fixo, o que facilita a manipulação de dados em tempo de execução.

9.1 Características

- ✿ Estrutura baseada em array interno
- ✿ Tamanho dinâmico
- ✿ Permite elementos duplicados
- ✿ Mantém a ordem de inserção
- ✿ Acesso rápido por índice
- ✿ Inserção/remoção pode ter custo maior (deslocamento de elementos)

Exemplo 9.1

```
1 import java.util.ArrayList;
2
3 public class ExemploArrayList {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         ArrayList<String> nomes = new ArrayList<>();
8
9         // Adicionando elementos
10        nomes.add("Ana");
11        nomes.add("Carlos");
12        nomes.add("Maria");
13
14        // Exibindo elementos
15        System.out.println("Lista de nomes:");
16        for (String nome : nomes) {
17            System.out.println(nome);
18        }
19    }
20 }
```

Também pode ser inicializado com capacidade inicial.

Exemplo 9.2

```
1 import java.util.ArrayList;
2
3 public class ExemploArrayListNumeros {
4     public static void main(String[] args) {
5
```

```

6      // Declaração e criação com capacidade inicial
7      ArrayList<Integer> numeros = new ArrayList<>(4);
8
9      // Adicionando elementos
10     numeros.add(10);
11     numeros.add(20);
12     numeros.add(30);
13     numeros.add(40);
14
15     // Exibindo os elementos
16     System.out.println("Números na lista:");
17     for (int num : numeros) {
18         System.out.println(num);
19     }
20
21     // Exibindo o tamanho atual da lista
22     System.out.println("Quantidade de elementos: " + numeros.size());
23 }
24 }

```

9.2 Métodos

Os principais métodos da classe `ArrayList` permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 9.1. Principais métodos da classe `ArrayList`

Método	Descrição
<code>add(E e)</code>	Adiciona um elemento ao final da lista.
<code>add(int index, E e)</code>	Insere um elemento em uma posição específica.
<code>get(int index)</code>	Retorna o elemento da posição informada.
<code>set(int index, E e)</code>	Substitui o elemento em uma posição específica.
<code>remove(int index)</code>	Remove o elemento pelo índice.
<code>remove(Object o)</code>	Remove a primeira ocorrência do elemento informado.
<code>size()</code>	Retorna a quantidade de elementos na lista.
<code>clear()</code>	Remove todos os elementos da lista.
<code>contains(Object o)</code>	Verifica se o elemento existe na lista.
<code>isEmpty()</code>	Verifica se a lista está vazia.

Exemplo 9.3

```
1 import java.util.ArrayList;
2
3 public class ExemploMetodosArrayList {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         ArrayList<String> nomes = new ArrayList<>();
8
9         // Adicionando elementos
10        nomes.add("Ana");
11        nomes.add("Carlos");
12        nomes.add("Maria");
13
14        // get: acessar elemento
15        System.out.println("Primeiro nome: " + nomes.get(0));
16
17        // set: alterar elemento
18        nomes.set(1, "João");
19
20        // contains: verificar existência
21        if (nomes.contains("Ana")) {
22            System.out.println("Ana está na lista");
23        }
24
25        // remove: remover elemento
26        nomes.remove("Maria");
27
28        // size: quantidade de elementos
29        System.out.println("Tamanho da lista: " + nomes.size());
30
31        // Exibindo lista atual
32        System.out.println("Lista: " + nomes);
33
34        // clear: remover todos os elementos
35        nomes.clear();
36
37        // isEmpty: verificar se está vazia
38        if (nomes.isEmpty()) {
39            System.out.println("A lista está vazia");
40        }
41    }
42 }
```


Capítulo 10 HashSet

O HashSet é uma classe da biblioteca `java.util` que implementa uma coleção baseada em tabela hash. Ele é utilizado para armazenar elementos de forma eficiente, sem permitir valores duplicados.

Diferente do `ArrayList`, o HashSet não mantém a ordem de inserção dos elementos.

10.1 Características

- ❖ Não permite elementos duplicados
- ❖ Não mantém ordem dos elementos
- ❖ Permite valores nulos (apenas um)
- ❖ Alta performance para inserção, remoção e busca

Exemplo 10.1

```
1 import java.util.HashSet;
2
3 public class ExemploHashSetCarros {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashSet<String> carros = new HashSet<>();
8
9         // Adicionando elementos
10        carros.add("Volvo");
11        carros.add("BMW");
12        carros.add("Ford");
13        carros.add("BMW"); // elemento duplicado (não será inserido)
14        carros.add("Mazda");
15
16        // Exibindo os elementos
17        System.out.println("Carros cadastrados:");
18        System.out.println(carros);
19
20        // Exibindo quantidade de elementos
21        System.out.println("Quantidade: " + carros.size());
22    }
23 }
```

10.2 Métodos

Os principais métodos da classe HashSet permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 10.1. Principais métodos da classe HashSet

Método	Descrição
add(E e)	Adiciona um elemento ao conjunto.
remove(Object o)	Remove o elemento informado.
contains(Object o)	Verifica se o elemento existe.
size()	Retorna a quantidade de elementos.
clear()	Remove todos os elementos.
isEmpty()	Verifica se o conjunto está vazio.

Exemplo 10.2

```

1 import java.util.HashSet;
2 public class ExemploHashSet {
3     public static void main(String[] args) {
4
5         // Declaração e criação
6         HashSet<String> nomes = new HashSet<>();
7
8         // Adicionando elementos
9         nomes.add("Ana");
10        nomes.add("Carlos");
11        nomes.add("Ana"); // duplicado (não será adicionado)
12
13        // Exibindo elementos
14        System.out.println("Elementos: " + nomes);
15
16        // Verificando elemento
17        if (nomes.contains("Carlos")) {
18            System.out.println("Carlos está no conjunto");
19        }
20
21        // Tamanho
22        System.out.println("Quantidade: " + nomes.size());
23
24        // Removendo elemento
25        nomes.remove("Ana");
26
27        // Verificando vazio
28        if (nomes.isEmpty()) {
29            System.out.println("Conjunto vazio");
30        } else {
31            System.out.println("Ainda possui elementos");
32        }
33    }
34 }

```

Capítulo 11 HashMap

O HashMap é uma classe da biblioteca `java.util` utilizada para armazenar dados no formato **chave-valor**. Cada chave é única e está associada a um valor.

Essa estrutura é muito utilizada quando precisamos realizar buscas rápidas com base em uma chave.

11.1 Características

- ✿ Armazena dados no formato chave - valor
- ✿ Não permite chaves duplicadas
- ✿ Permite valores duplicados
- ✿ Não mantém ordem dos elementos
- ✿ Alta eficiência para busca e inserção

Exemplo 11.1

```
1 import java.util.HashMap;
2
3 public class ExemploHashMapPaises {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashMap<String, String> capitais = new HashMap<>();
8
9         // Adicionando elementos (País, Capital)
10        capitais.put("Inglaterra", "Londres");
11        capitais.put("Índia", "Nova Délhi");
12        capitais.put("Áustria", "Viena");
13        capitais.put("Noruega", "Oslo");
14        capitais.put("Noruega", "Oslo"); // chave duplicada (não será adicionada
15        novamente)
16        capitais.put("EUA", "Washington DC");
17
18        // Exibindo os dados
19        System.out.println("Capitais cadastradas:");
20        System.out.println(capitais);
21
22        // Exibindo quantidade de elementos
23        System.out.println("Quantidade: " + capitais.size());
24    }
```

11.2 Métodos

Os principais métodos da classe HashMap permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 11.1. Principais métodos da classe HashMap

Método	Descrição
put(K key, V value)	Adiciona ou atualiza um par chave-valor.
get(Object key)	Retorna o valor associado à chave.
remove(Object key)	Remove o elemento pela chave.
containsKey(Object key)	Verifica se a chave existe.
containsValue(Object value)	Verifica se o valor existe.
size()	Retorna a quantidade de elementos.
clear()	Remove todos os elementos.
isEmpty()	Verifica se o mapa está vazio.

Exemplo 11.2

```

1 import java.util.HashMap;
2
3 public class ExemploHashMap {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashMap<String, Integer> alunos = new HashMap<>();
8
9         // Adicionando elementos (chave, valor)
10        alunos.put("Ana", 8);
11        alunos.put("Carlos", 9);
12        alunos.put("Maria", 7);
13
14        // Acessando valor pela chave
15        System.out.println("Nota de Ana: " + alunos.get("Ana"));
16
17        // Verificando chave
18        if (alunos.containsKey("Carlos")) {
19            System.out.println("Carlos está cadastrado");
20        }
21
22        // Exibindo quantidade
23        System.out.println("Quantidade: " + alunos.size());
24
25        // Removendo elemento
26        alunos.remove("Maria");
27
28        // Exibindo estrutura
29        System.out.println(alunos);
30    }
31 }

```

Capítulo 12 Fila

A fila (Queue) é uma interface da biblioteca `java.util` utilizada para armazenar elementos seguindo a lógica **FIFO** (First In, First Out), ou seja, o primeiro elemento inserido é o primeiro a ser removido.

Ela é muito utilizada em situações como filas de atendimento, processamento de tarefas e controle de execução.

12.1 Características

- ✿ Estrutura do tipo fila (FIFO)
- ✿ Permite inserção no final e remoção no início
- ✿ Não permite acesso direto por índice
- ✿ Possui diferentes implementações (ex: `LinkedList`, `PriorityQueue`)

Como fila é uma interface, utilizamos uma classe que a implementa, como `LinkedList`:

Exemplo 12.1

```
1 import java.util.Queue;
2 import java.util.LinkedList;
3
4 public class ExemploQueue {
5     public static void main(String[] args) {
6
7         // Declaração e criação
8         Queue<String> fila = new LinkedList<>();
9
10        // Adicionando elementos
11        fila.add("Ana");
12        fila.add("Carlos");
13        fila.add("Maria");
14
15        // Exibindo fila
16        System.out.println("Fila: " + fila);
17
18        // Consultando o primeiro elemento
19        System.out.println("Primeiro da fila: " + fila.peek());
20
21        // Removendo elemento
22        fila.poll();
23
24        // Exibindo após remoção
25        System.out.println("Fila atualizada: " + fila);
26    }
27 }
```

12.2 Métodos

Os principais métodos da fila permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 12.1. Principais métodos da interface Queue

Método	Descrição
<code>add(E e)</code>	Adiciona elemento ao final da fila.
<code>offer(E e)</code>	Adiciona elemento (sem lançar erro se falhar).
<code>remove()</code>	Remove e retorna o primeiro elemento.
<code>poll()</code>	Remove e retorna o primeiro elemento (ou null).
<code>element()</code>	Retorna o primeiro elemento sem remover.
<code>peek()</code>	Retorna o primeiro elemento (ou null).

12.3 Exemplo

Exemplo 12.2

```
1 import java.util.Queue;
2 import java.util.LinkedList;
3
4 public class ExemploQueue {
5     public static void main(String[] args) {
6
7         // Declaração e criação
8         Queue<String> fila = new LinkedList<>();
9
10        // Adicionando elementos
11        fila.add("Ana");
12        fila.add("Carlos");
13        fila.add("Maria");
14
15        // Exibindo fila
16        System.out.println("Fila: " + fila);
17
18        // Consultando o primeiro elemento
19        System.out.println("Primeiro: " + fila.peek());
20
21        // Removendo elemento
22        fila.poll();
23
24        // Exibindo após remoção
25        System.out.println("Fila atualizada: " + fila);
26    }
27 }
```

Capítulo 13 Stack

A pilha, Stack é uma classe da biblioteca `java.util` que representa uma estrutura de dados baseada no princípio **LIFO** (Last In, First Out), ou seja, o último elemento inserido é o primeiro a ser removido.

Essa estrutura é muito utilizada em situações como controle de chamadas de funções, desfazer ações (undo) e processamento de expressões.

13.1 Características

- ✿ Estrutura do tipo pilha (LIFO)
- ✿ Inserção e remoção no topo da pilha
- ✿ Permite acesso ao elemento do topo
- ✿ Baseada na classe `Vector`

Exemplo 13.1

```
1 import java.util.Stack;
2
3 public class ExemploStack {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         Stack<String> pilha = new Stack<>();
8
9         // Adicionando elementos
10        pilha.push("Ana");
11        pilha.push("Carlos");
12        pilha.push("Maria");
13
14        // Exibindo pilha
15        System.out.println("Pilha: " + pilha);
16
17        // Consultando o topo
18        System.out.println("Topo da pilha: " + pilha.peek());
19
20        // Removendo elemento do topo
21        pilha.pop();
22
23        // Exibindo após remoção
24        System.out.println("Pilha atualizada: " + pilha);
25    }
26 }
```

13.2 Métodos

Os principais métodos da classe pilha `Stack` permitem manipular os elementos da pilha, incluindo operações de inserção, remoção e consulta do topo.

Tabela 13.1. Principais métodos da classe Stack

Método	Descrição
push(E item)	Adiciona elemento ao topo da pilha.
pop()	Remove e retorna o elemento do topo.
peek()	Retorna o elemento do topo sem remover.
empty()	Verifica se a pilha está vazia.
search(Object o)	Retorna a posição do elemento na pilha.

Exemplo 13.2

```
1 import java.util.Stack;
2
3 public class ExemploStack {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         Stack<String> pilha = new Stack<>();
8
9         // Adicionando elementos
10        pilha.push("Ana");
11        pilha.push("Carlos");
12        pilha.push("Maria");
13
14        // Exibindo pilha
15        System.out.println("Pilha: " + pilha);
16
17        // Consultando o topo
18        System.out.println("Topo: " + pilha.peek());
19
20        // Removendo elemento do topo
21        pilha.pop();
22
23        // Exibindo após remoção
24        System.out.println("Pilha atualizada: " + pilha);
25    }
26 }
```

Capítulo 14 Exercícios

Exercício 14.1

Cadastro de Cliente - Crie um programa em Java que solicite os seguintes dados do cliente:

- ✿ Nome
- ✿ Idade
- ✿ Salário mensal
- ✿ Valor do empréstimo desejado

Ao final, exiba todas as informações digitadas em uma caixa de mensagem. Use JOptionPane para receber os dados.

Exercício 14.2

Análise de Crédito Simples - Desenvolva um programa em Java para realizar uma análise simples de crédito. O programa deve solicitar os seguintes dados do cliente: **(i) Nome; (ii) Salário mensal; (iii) Valor da parcela desejada.**

Considere a seguinte regra de análise. Se o valor da parcela for menor ou igual a 30% do salário, o crédito pode ser analisado. Caso contrário, o crédito deve ser negado. Ao final, o sistema deve exibir: (i) Nome do cliente; (ii) Salário mensal; (iii) Valor da parcela desejada; (iv) Resultado da análise.

Objetivo: aplicar cálculo percentual, estrutura condicional composta e uso de componentes da interface gráfica Swing. Use elementos da interface gráfica Swing (por exemplo: showInputDialog, showConfirmDialog, showOptionDialog)

Exercício 14.3

Classificação de risco - Crie um programa que leia: (i) Nome do cliente; (ii) Salário mensal; (iii) Score de crédito e classifique o cliente da seguinte forma: Score acima de 800: risco baixo, Score entre 600 e 800: risco médio, e Score abaixo de 600: risco alto. Exiba a classificação em uma mensagem.

Objetivo: praticar decisões encadeadas.

Exercício 14.4

Sistema simples de aprovação de Crédito - Crie um programa que receba: (i) Nome, (ii) Idade, (iii) Salário, (iv) Valor do empréstimo desejado, (v) Número de parcelas. Após isso ele vai calcular automaticamente o valor da parcela, usando a conta

$$\text{parcela} = \frac{\text{valor do empréstimo}}{\text{número de parcelas}}$$

Depois o algoritmo deve aplicar as regras para aprovação de crédito:

- ✿ Idade mínima de 18 anos
- ✿ Salário maior que 0
- ✿ Parcela menor ou igual a 30% do salário

Se todas as condições forem atendidas, exiba **“Crédito aprovado”**. Caso contrário, exiba **“Crédito negado”**.

Objetivo: Integrar entrada de dados, processamento matemático e múltiplas condições lógicas.